Cryptographic Admissibility and Jurisdiction

CHC as a Mathematical Primitive for State Identity, Semantic Identity, and Admissible Authority

The state hash binds the jurisdiction. The semantic hash binds the claims. Admissibility decides what may act.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Cryptographic verification systems typically bind artifacts to hashes, signatures, attestations, or provenance records. These mechanisms can often prove that a file, binary, package, document, or runtime artifact has not changed, but they do not natively bind that artifact to typed claims about intent, authority, jurisdiction, custody, resource use, lineage, or present admissibility under policy.

Cryptographic Homological Compilation (CHC) defines a two-hash architecture and a freshness verification procedure. The State Hash H_s binds the artifact. The Semantic Hash H_m binds typed semantic dimensions whose values and attestations are independently verifiable under declared policy. Freshness is an evaluation layer, not a hash object.

CHC's novelty is not hashes, signatures, Merkle trees, attestations, provenance, freshness, or revocation in isolation. Its contribution is the separation of state identity, semantic identity, and admissibility into independently addressable cryptographic domains. Traditional cryptographic systems bind state. CHC binds state, claims, and admissibility as separately addressable cryptographic domains. The resulting architecture replaces authority-by-assertion with authority-by-admissibility, where signatures, institutions, registries, and attestations become evidence inputs rather than self-authorizing sources of legitimacy.

CHC does not prove meaning. It cryptographically binds attestations about meaning to artifact state, then evaluates whether those bindings remain admissible under declared policy. Freshness evaluates admissibility, not truth. More generally, CHC relocates authority from implicit institutional trust toward explicit policy-evaluable admissibility. Signatures, attestations, manifests, checkpoints, and other evidence sources become inputs to policy rather than self-justifying sources of authority. The result is a candidate architectural foundation for a family of governed execution systems, including verified compilation,

supply-chain security, distributed identity, constitutional governance systems, semantic operating systems, and continuous execution governance.

Security boundary: CHC binds attestations about meaning to artifact state and evaluates admissibility under policy; it does not establish the truth of the claims themselves.

---

1. Introduction

A conventional cryptographic hash proves that an artifact is stable.

A signature proves that a key endorsed something.

An attestation proves that a claim was made under some procedure.

But software execution depends on more than artifact stability, signer identity, or procedural claims. A binary may be bit-identical and still stale. A manifest may be signed and still superseded. A source-to-binary claim may be present and still lack authority. A package may carry provenance and still be inadmissible under a local policy.

CHC is not primarily a software distribution protocol. It is a cryptographic admissibility primitive from which software distribution, supply-chain governance, semantic identity systems, constitutional runtimes, and related architectures may be constructed.

More precisely, CHC is an implementation architecture of Cryptographic Admissibility, not vice versa. Cryptographic Admissibility is the theory. CHC is the first binding architecture that realizes the theory through State Hashes, Semantic Hashes, receipts, freshness evaluation, and policy-bound admissibility.

The missing primitive is a cryptographic structure that separates:

```
state integrity
semantic attestation
present admissibility under policy
```

Existing verification systems often conflate evidence production with authority. A signature, transparency log, registry entry, or institutional attestation may be treated as implicitly authoritative because of its source. CHC instead treats all such outputs as evidence objects whose admissibility is determined by explicit policy. Authority becomes a property of evaluation rather than origin.

CHC proposes that separation through:

```
H_s -> H_m -> Freshness Evaluation
```

where:

- H_s binds a deterministic State Jurisdiction;
- H_m binds typed semantic claims to that state;

- freshness evaluation determines whether the state-and-semantic binding may still be accepted.

This structure prevents three common collapses:

1. treating identity as authority;
2. treating historical validity as present admissibility;
3. treating metadata as verified semantic content.

The broader contribution is a generalization of content addressing. Traditional content addressing asks:

```
What exists?
```

CHC asks three separable questions:

```
What exists?
What is claimed?
What may act?
```

This turns content addressing into admissibility addressing.

---

2. Notation

Symbol	Meaning
J	State Jurisdiction space
j	State Jurisdiction
A	Artifact space, a common subclass of State Jurisdictions
a	Artifact
D	Typed semantic dimension space
$\mathcal{D}$	Schema-ordered tuple of semantic dimensions
M	Manifest sources and manifest state
R	Trust-root set
E	Evidence set
P	Deterministic policy program
t	Verifiable epoch, checkpoint, or log position
π	Freshness, consistency, inclusion, or revocation evidence
ρ	Receipt
$\mathcal{R}$	Receipt graph
H_s	State Hash
H_m	Semantic Hash
H_R	Policy-bound digest of the active trust-root set
H_P	Policy-bound digest of the active policy program
H_M	Manifest root hash

H_R and H_P are operational inputs to freshness evaluation. A verifier must know

not only which roots and policy exist, but which root-set interpretation and policy version are active for the decision.

---

3. Threat Model

The adversary may forge metadata, substitute manifests, equivocate between manifests, replay stale attestations, compromise a signer, suppress revocation information, exploit schema ambiguity, spoof registries, roll back schemas, present old but valid binaries, downgrade policy, exploit policy-source inconsistency, poison caches, or attempt to mix valid state hashes with unrelated semantic claims.

CHC's security claims hold only under declared assumptions: collision-resistant and second-preimage-resistant hash functions, EUF-CMA-secure signatures, canonical encoding, authenticated trust-root state, authenticated schema registry state, policy-correct verification, and the availability assumptions required by the selected freshness sources.

Claims about compiler subversion, supply-chain integrity, or execution safety are bounded by the declared CHC verification boundary. Components outside that boundary remain trusted premises unless modeled as CHC State Jurisdictions with their own semantic attestations, receipts, and freshness checks.

---

4. Formal Model

4.1 State Jurisdiction

Let J be the State Jurisdiction space.

A State Jurisdiction is a deterministic, policy-declared boundary over which state identity is computed.

Examples include:

- a source file;
- a binary;
- a package;
- a repository;
- a container image;
- a model checkpoint;
- a runtime state;
- a deployment snapshot;
- a manifest tree;
- a distributed state graph.

Artifacts are one class of State Jurisdiction:

$$A \subseteq J$$

The State Hash is:

$$H_s : J \to \{0,1\}^n$$

with:

$$h_s = H_s(j)$$

The State Hash does not prescribe the granularity of state. It fingerprints a declared State Jurisdiction. Multiple jurisdictions may coexist simultaneously over the same underlying materials. CHC standardizes the fingerprinting relationship rather than a specific level of granularity.

H_s is computed from the bytes or canonical state representation of j after applying the declared normalization rule. If normalization depends on state-type metadata outside the bytes themselves, that metadata is part of the hash domain and must be bound in the Artifact State and Encoding dimension. For native binary builds, the normalization boundary is the clean pre-metadata artifact produced by Phase 1. For containers, documents, repositories, deployment snapshots, and structured packages, the normalization boundary is the schema-defined and versioned canonical serialization of the state payload, excluding CHC metadata unless the schema explicitly states otherwise. The normalization boundary is therefore a compatibility contract.

H_s must not depend on semantic dimensions, manifests, policies, freshness state, or attestations.

4.2 Jurisdiction Algebra

State Jurisdictions may relate to one another.

A file may belong to a repository. A repository may belong to a release. A release may belong to a distribution. A compiler may belong to a toolchain. A toolchain may belong to a build environment.

CHC therefore defines jurisdiction relationships:

$$J_1 \subseteq J_2$$

meaning that J_1 is contained within the state boundary of J_2, and:

$$derive(J_1 \to J_2)$$

meaning that the identity or admissibility of J_2 depends on a declared relationship to J_1.

Jurisdiction composition is policy-defined:

$$compose(J_1, J_2) \to J_3 \cup Conflict$$

Composition succeeds only when the two jurisdictions have compatible normalization rules, schema versions, and policy-declared relation types. Otherwise, composition returns a conflict object.

Examples:

```
source file
  in repository
      in release
          in distribution

compiler binary
  in toolchain
      in build environment
          in deployment pipeline

knowledge shard
  in knowledge graph
      in distributed knowledge network
```

The purpose of jurisdiction algebra is not to impose one global hierarchy. It is to make jurisdictional relationships explicit, hashable, and policy-evaluable.

4.3 Typed Semantic Dimension Space

Let the semantic dimension space be a disjoint typed union:

$$D = \bigsqcup_{i \in I} D_i$$

Each D_i is a typed domain registered under a dimension identifier.

A dimension instance is:

$$d_i = \langle id_i, schema_i, version_i, value_i, \\ attestation_i \rangle \in D_i$$

The dimension collection attached to a State Jurisdiction is a schema-ordered tuple:

$$\mathcal{D} = (d_{(1)}, d_{(2)}, \ldots, d_{(k)})$$

$\mathcal{D}$ has no author-controlled ordering. Its order is determined by the schema registry. Duplicate dimensions are forbidden unless the schema explicitly declares a multivalue rule.

Empty $\mathcal{D}$ is a degenerate case valid only for non-executable, non-distributable, non-deployable artifact classes whose policy explicitly permits no semantic claims. Executable, distributable, deployable, or publishable jurisdictions require policy-defined core dimensions.

4.4 Well-Formedness

A dimension tuple is admissible for hashing only if

$$WF(\mathcal{D}) = \text{true}$$

where WF requires:

1. every id_i resolves in the schema registry;
2. every schema is hash-addressed, signed or transparency-logged, and versioned;
3. every value is normalized under its schema;
4. no implicit defaults are used;
5. every required core dimension is present;
6. every attestation cryptographically binds h_{v_i}, H_s, schema_hash$_i$, signer identity, issuer or signer class, policy context, and expiry or validity interval;
7. the same expiry or validity interval is visible to the policy state so freshness can reject claims whose signatures remain cryptographically valid but whose admissibility interval has expired;
8. every duplicate or multivalue dimension obeys its schema rule.

Normalization rules must be explicit for semantically equivalent but byte-distinct values, including timestamps, URIs, identifiers, JSON-like objects, numeric encodings, and Unicode strings.

4.5 Governance Algebra

Let G denote the algebra of operations available over semantic dimension tuples and their evidence objects.

Projection is an operation:

$$proj_S : \mathcal{D} \to S$$

which extracts a policy-declared semantic view.

Merge is an operation:

$$merge : \mathcal{D}_1 \times \mathcal{D}_2 \to \mathcal{D}_3 \cup Conflict$$

Merge fails unless a policy-defined reconciliation event is present. A failed merge returns a conflict object rather than silently choosing a branch.

Other operations include dimension extension, revocation application, trust-root selection, manifest lookup, freshness evaluation, receipt graph traversal, jurisdiction composition, and policy execution.

A policy predicate is a deterministic program drawn from that algebra:

$$g_P : (H_s, \mathcal{D}, R, M, t, \pi) \to \{\text{true}, \text{false}\}$$

CHC does not assume that governance predicates are monotone. A policy must explicitly declare whether adding dimensions preserves, narrows, or invalidates admissibility. The algebra supports extension, but no policy may silently treat extension as safe.

4.6 Admissibility Principle

CHC treats authority as a derived property of admissibility rather than a primitive property of identity.

Signatures, receipts, manifests, checkpoints, witnesses, institutions, and registries contribute evidence. Policy determines admissibility.

```
Identity != Authority
```

```
Authority := Admissibility(Evidence, Policy)
```

Authority is not a primitive property. Authority emerges from admissible evidence evaluated under declared policy.

$$\text{Authority}(E, P) := \text{Admissible}(E, P)$$

In CHC, an entity does not become authoritative merely because it signs, publishes, witnesses, or stores a claim. Its output becomes evidence. Whether that evidence has authority is determined by the active policy over semantic dimensions, jurisdiction, receipts, freshness state, and trust-root interpretation.

This is the central governance inversion:

```
authority-by-assertion
        ->
authority-by-admissibility
```

Under authority-by-admissibility, signatures, institutions, registries, transparency systems, and witnesses become evidence producers rather than final arbiters of authority.

4.7 Identity–Authority Separation Principle

Any architecture that permits authority mutation without state mutation must represent state identity and authority as independently addressable domains.

Equivalently:

> If a system allows authority claims to change while preserving the identity of the underlying state, then authority cannot be identical to state identity.

Proof sketch. Authority may change under policy, revocation, freshness, or jurisdictional reinterpretation while H_s remains invariant; therefore authority is not identical to state identity. A formal proof of this principle, under explicit definitions of architectural addressability and authority mutation, is left to subsequent work.

CHC satisfies this principle by separating:

```
H_s = state identity

H_m = semantic identity

P   = admissibility evaluation
```

Thus authority is not carried by H_s alone, nor by signer identity alone, nor by the mere existence of an attestation. Authority is derived when policy evaluates semantic claims, receipts, jurisdiction, freshness, and trust-root state as admissible.

This principle is the formal basis for CHC's authority normalization: the signer, institution, registry, or manifest does not become authority by assertion. It becomes evidence within an admissibility calculus.

---

5. The Cryptographic Admissibility Triad

CHC separates cryptographic identity into three independently evaluable domains:

```
State Identity
      down
Semantic Identity
      down
Admissibility
```

The corresponding operational structure is:

```
H_s = State Identity

H_m = Semantic Identity

P = Admissibility
```

State identity answers:

> What exists within this State Jurisdiction?

Semantic identity answers:

> What is claimed about that state?

Admissibility answers:

> What may act under policy?

Traditional systems often collapse these layers. CHC separates them.

This separation is the core architecture of cryptographic admissibility.

5.1 Admissibility Separation Principle

State identity, semantic identity, and admissibility are independently mutable domains.

A valid cryptographic governance architecture should permit change in one domain without requiring mutation of the others.

Formally:

```
State Identity      = H_s(J)

Semantic Identity   = H_m(h_s, D, R, P)

Admissibility       = P(H_s, H_m, D, M, R, t, pi)
```

A change to semantic claims should not require mutation of the underlying state.

A change to policy should not require mutation of the state or semantic hash.

A change to freshness evidence should not require mutation of state identity or semantic identity.

This is the separation that allows CHC to distinguish:

```
what exists
what is claimed
what may act
```

The principle can be stated compactly:

> State identity, semantic identity, and admissibility must be separately addressable so that authority claims may change without rewriting state, and admissibility may change without falsifying prior claims.

This is the Cryptographic Admissibility Principle.

5.2 Separation Consequence

The Admissibility Separation Principle implies that CHC can refuse execution without denying artifact integrity.

It can revoke admissibility without rewriting history.

It can update policy without mutating state identity.

It can add semantic claims without changing the State Hash.

It can preserve evidence while denying authority.

This is the practical distinction between:

```
the claim was made
```

and:

```
the claim may act
```

5.3 Architectural Diagram

```
State Jurisdiction
        down
      H_s
        down
  State Identity
        down
Semantic Dimensions
        down
      H_m
        down
 Semantic Identity
        down
```

```
Receipts / Evidence
        down
Freshness Evaluation
        down
 Admissibility
        down
Execution / Refusal Decision
```

The diagram should be read as a separation of addressable domains, not as a single linear pipeline. State identity may remain stable while semantic identity changes. Semantic identity may remain historically valid while admissibility changes. Admissibility may change because policy, receipts, freshness evidence, revocation state, or jurisdictional interpretation changes.

This separation may be compressed into a single invariant: the three domains are independently addressable.

5.4 CHC in One Sentence

Traditional cryptographic systems bind state.

CHC binds state, claims, and admissibility as separately addressable cryptographic domains.

---

6. CHC Novelty Boundary

CHC does not invent:

```
hashes
signatures
Merkle trees
attestations
provenance
freshness
revocation
```

CHC introduces and composes the following primitives:

```
State Jurisdiction

Semantic Identity

Admissibility Separation

Authority Normalization
```

`Two-Hash Binding`

`Cryptographic Admissibility`

The novelty is not that CHC uses cryptographic evidence.

CHC's novelty claim is not the existence of any individual component but the formal treatment of State Identity, Semantic Identity, and Admissibility as independently addressable cryptographic domains. Existing systems may provide identity, provenance, attestation, freshness, or policy independently; CHC treats semantic identity as a first-class cryptographic object positioned between state identity and admissibility. A Semantic Hash differs from metadata hashing because dimensions are schema-governed, independently attestable, policy-addressable claims rather than opaque auxiliary fields.

---

7. The Two-Hash Rule

CHC requires two independent cryptographic layers.

7.1 State Hash

$$H_s$$

answers:

> What state jurisdiction exists?

7.2 Semantic Hash

$$H_m$$

answers:

> What typed attestations are bound to this state jurisdiction?

7.3 State Jurisdiction Principle

The State Hash does not prescribe the granularity of state. It fingerprints a declared State Jurisdiction. A jurisdiction may consist of a single file, a binary, a repository, a manifest tree, a deployment snapshot, a distributed state graph, or any other deterministically bounded state domain. Multiple jurisdictions may coexist simultaneously. CHC therefore standardizes the fingerprinting relationship rather than a specific level of granularity.

7.4 Freshness Evaluation

Freshness is not a third hash.

Freshness answers:

> Is this state-and-semantic binding still admissible now under the active policy hash and active trust-root hash?

It is policy-relative and time-relative. It is evaluated over:

$$H_s,\ H_m,\ \mathcal{D},\ M,\ R,\ P,\ t,\ \pi$$

A separate freshness digest would merely package the result of a verification procedure and would itself require freshness evaluation. CHC therefore keeps the cryptographic basis minimal:

$$H_s \to H_m$$

with freshness performed at the point of use.

7.5 Why Not One Hash?

One hash collapses state identity and semantic claims into one digest. Any change to authority, jurisdiction, lineage, custody, schema, or attestation would require treating the underlying state as changed.

This destroys separation of concerns.

7.6 Why Not One State Hash Plus Externally Evaluated Freshness Evidence?

A state-hash-plus-freshness-evidence system can check whether a state is current, and that evidence may itself be manifest-bound, checkpoint-bound, or log-backed. But it cannot distinguish current state integrity from current semantic admissibility. It lacks a cryptographic object binding typed semantic claims to state.

The minimal structure is therefore:

```
state hash
semantic hash
freshness procedure
```

not:

```
state hash only
```

and not:

`state hash + external freshness evidence`

---

8. Domain Separation

CHC uses explicit domain separation tags.

$$H_s(j) = H(\texttt{"CHC:H_s:v1"} \parallel Encode_J(j))$$

$$H_m(h_s, \mathcal{D}, H_R, H_P) = H(\texttt{"CHC:H_m:v1"} \parallel h_s \parallel H_R \parallel H_P \parallel E(\mathcal{D}))$$

Each dimension value hash uses its own tag:

$$h_{v_i} = H(\texttt{"CHC:DIMVALUE:v1"} \parallel value_i)$$

Each attestation message uses its own tag:

$$msg_i = \texttt{"CHC:DIMATT:v1"} \parallel ...$$

Domain separation is mandatory across state hash, semantic hash, value hash, attestation message, policy hash, manifest hash, and root-set hash. A digest valid in one layer must not be reusable in another.

H_R is a digest of the trust-root set. It is not a substitute for policy evaluation, revocation state, or freshness checking.

---

9. Semantic Hash Construction

The Semantic Hash is:

$$H_m : (h_s, \mathcal{D}, H_R, H_P) \to \{0,1\}^n$$

defined as:

$$H_m = H(\texttt{"CHC:H_m:v1"} \parallel H_s(j) \parallel H_R \parallel H_P \parallel E(\mathcal{D}))$$

H_m is binding over the canonical encoding $(h_s, E(\mathcal{D}), H_R, H_P)$ under standard hash assumptions.

CHC does not claim that H_m authenticates meaning. It binds attestations about meaning to state identity.

Optional dimensions are included in H_m only when present and schema-admissible. Omitted optional dimensions are represented as absence, not as implicit nulls, unless the schema explicitly defines a null-valued dimension. When present, optional dimensions change H_m. Optionality affects policy evaluation, not hash inclusion.

Dimension extension must preserve second-preimage resistance. Given:

$$\mathcal{D}' = \mathcal{D} \cup \{d_{k+1}\}$$

it should be computationally infeasible to find:

$$(H_s, \mathcal{D}, H_R, H_P) \neq (H'_s, \mathcal{D}', H'_R, H'_P)$$

such that both produce the same semantic hash, except with negligible probability.

---

10. Canonical Ordering

Each schema registry entry has:

$$registry_key_i = H(id_i \parallel schema_hash_i \parallel version_i)$$

Dimensions are ordered lexicographically by:

$$(registry_key_i,\ id_i,\ version_i)$$

The schema hash appears twice by design as defense-in-depth: once for registry ordering and once to bind each value and attestation to its interpretation schema.

---

11. Dimension Encoding

Each dimension d_i is encoded as:

$$
\begin{aligned}
E(d_i) = \; & len(id_i) \\
& \| \; id_i \\
& \| \; len(schema_i) \\
& \| \; schema_i \\
& \| \; len(version_i) \\
& \| \; version_i \\
& \| \; len(value_i) \\
& \| \; value_i \\
& \| \; len(h_{v_i}) \\
& \| \; h_{v_i} \\
& \| \; len(attestation_i) \\
& \| \; attestation_i
\end{aligned}
$$

Encoding must use a deterministic canonical format such as deterministic CBOR, SSZ, or another declared canonical serialization. Canonical serialization must be byte-for-byte deterministic across implementations. Implicit defaults are forbidden.

Length-prefixing alone is insufficient unless nested encodings also have unambiguous field delimiters and canonical field ordering.

The dimension tuple encoding is:

$$
\begin{aligned}
E(\mathcal{D}) = \; & len(k) \\
& \| \; E(d_{(1)}) \\
& \| \; \cdots \\
& \| \; E(d_{(k)})
\end{aligned}
$$

All variable-length fields are length-prefixed.

---

12. Receipts

A receipt is a cryptographically accountable evidence object recording a claim, observation, or transition relevant to a State Jurisdiction. Receipts are the primary evidentiary substrate through which semantic identity participates in admissibility evaluation.

A receipt records at minimum:

```
receipt = {
  observer_or_issuer,
  state_jurisdiction,
```

```
  state_hash,
  semantic_claim_or_event,
  policy_context,
  epoch_or_checkpoint,
  evidence_payload,
  attestation_or_signature,
  receipt_schema
}
```

Receipts are the primary evidentiary substrate of CHC. Logs are implementation artifacts; receipts are admissibility artifacts. They may support dimension attestations, freshness checks, manifest consistency, custody derivation, causal lineage, quorum finality, checkpoint continuity, or policy evaluation.

A receipt does not automatically confer authority. It contributes evidence to the admissibility predicate.

---

13. Receipt Graphs

Receipts may compose into a Receipt Graph:

$$\mathcal{R} = (V_R, E_R)$$

where:

- V_R is a set of receipts;
- E_R is a set of receipt dependencies;
- each edge records that one receipt relies on, confirms, supersedes, refutes, or derives from another receipt.

A Receipt Graph must be acyclic unless the schema explicitly defines a cycle-safe consensus or CRDT interpretation.

Receipt graphs provide the evidentiary substrate for provenance, custody, recursive toolchain binding, freshness continuity, and semantic lineage.

Example:

```
Receipt A
   down
Receipt B
   down
Receipt C
```

A Receipt Graph answers:

> What evidence depends on what prior evidence?

This gives CHC a cryptographic provenance structure without treating any single receipt as self-authorizing.

---

14. Dimension Attestation

An attestation is any accountable cryptographic evidence object capable of binding a semantic claim to a State Hash under a declared State Jurisdiction.

For each dimension d_i, the attested message is:

$$
\begin{aligned}
msg_i = {} & \texttt{"CHC:DIMATT:v1"} \\
& \| \, id_i \\
& \| \, schema_hash_i \\
& \| \, version_i \\
& \| \, h_{v_i} \\
& \| \, H_s \\
& \| \, H_P \\
& \| \, policy_context_i \\
& \| \, signer_identity_i \\
& \| \, signer_class_i \\
& \| \, validity_interval_i
\end{aligned}
$$

Signer identities must be normalized under a schema-declared identity encoding before inclusion in attestation messages. Equivalent signer identities expressed in different formats must either canonicalize to the same byte representation or fail comparison.

Attestation forms include:

- single signature;
- multisignature;
- threshold proof;
- aggregate signature;
- quorum receipt;
- checkpoint receipt;
- consensus receipt;
- witness receipt.

Signers may be witnesses, builders, custodians, auditors, issuers, or threshold members. Authority must still be separately established by predicate. Signer identity alone is not authority.

Validity intervals are mandatory for dimensions that affect authority, revocation, or execution admissibility.

15. Signature and Receipt Verification

For a single signature:

$$VerifySingle_i(d_i, R, P) = \text{true}$$

iff the signature verifies over msg_i, the signer is valid under R, and the signer is permitted by policy to attest dimension D_i.

Threshold, multisignature, aggregate-signature, receipt, checkpoint, and quorum schemes are schema-specific and not interchangeable unless the schema explicitly declares equivalence. The schema registry must distinguish aggregate proof mode from threshold proof mode and receipt verification mode.

For aggregate, threshold, quorum, or receipt-based attestations:

$$VerifyEvidence_i(d_i, R, P, q_i) = \text{true}$$

iff:

1. the evidence object verifies under its schema;
2. signer-set, witness-set, or authority-set membership proofs verify where required;
3. the valid set satisfies the policy threshold or receipt rule.

Aggregate validity alone is insufficient unless the scheme cryptographically binds the signer set and the policy accepts that binding.

The dimension verifier is:

$$Verify_i = VerifySingle_i \vee VerifyEvidence_i$$

depending on the schema-declared attestation mode.

The full semantic verification rule is:

$$V(H_m) = WF(\mathcal{D}) \wedge \bigwedge_{i=1}^{k} Verify_i(d_i, R, P)$$

16. Trust Roots

Let:

$$R = \{r_1, \ldots, r_m\}$$

be the trust-root set.

Each root contains:

- root identifier;
- public key or verification rule;
- jurisdiction;
- validity interval;
- revocation rule;
- rotation rule;
- policy binding.

The trust-root hash is:

$$H_R = H(\texttt{"CHC:ROOTSET:v1"} \parallel E(R))$$

The trust-root hash commits to the root set and its canonical ordering or normalization rule.

Trust roots establish admissible verification pathways.

Trust roots are evidence-routing mechanisms, not authority sources. Authority remains policy-derived. They do not themselves establish authority. Authority remains a policy-evaluated property derived from semantic dimensions, receipts, jurisdiction, freshness state, and active policy.

Trust-root set hash and active policy hash are both inputs to trust-root validation. Root material alone is insufficient without the policy that interprets it.

Trust-root models include PKI, web-of-trust, threshold governance, append-only transparency logs, and constitutional governance systems.

Root rotation and revocation are signed updates over R, producing a new H_R. Different trust domains may reach different admissibility decisions over the same state. CHC makes this explicit rather than pretending that trust is universal.

---

17. Canonical Governance Dimensions

CHC defines eight canonical governance dimensions.

Mandatory Core

1. **Artifact State and Encoding** — artifact identity, media type, serialization format, normalization rule, artifact role, and H_s;
2. **Causal** — parent state, parent semantic hash, or causal predecessor;
3. **Intent** — declared purpose or action;
4. **Authority** — predicate-bearing authority claim;
5. **Jurisdiction** — policy domain in which authority is evaluated;
6. **Custody** — actor, agent, steward, or custody chain.

Canonical Optional

7. **Semantics** — behavioral contract or semantic interpretation, when applicable;
8. **Resources** — observed or declared resource use.

The canonical model is (8+n), where (n) denotes extension dimensions. Some deployments may record Resources as observable metadata rather than a semantic-hash envelope member, but the dimension remains part of the canonical governance model.

Confidence is not part of the canonical eight. It may be introduced as an extension dimension with its own schema.

---

18. Artifact State and Encoding Dimension

The Artifact State and Encoding dimension has the following normative schema sketch:

```
artifact_state_and_encoding = {
  required state_hash: H_s,
  required media_type: string,
  required serialization_format: string,
  required normalization_scheme: string,
  required artifact_role: enum[
    source,
    build_input,
    intermediate,
    clean_binary,
    release_binary,
    package,
    container_image,
    policy_object,
    manifest_object,
    runtime_state
  ],
```

```
  optional artifact_size: integer,
  optional content_address: string
}
```

This prevents ambiguity about what was hashed.

---

19. Causal Dimension

The Causal dimension is a relation over hashes:

$$Causal \subseteq Hash \times Hash \times RelationType$$

A causal edge is:

$$(\text{parent_hash},\ \text{child_hash},\ \text{relation})$$

Causal edges may represent immediate predecessors or transitive ancestry. If transitive ancestry is stored, the schema must declare whether the stored relation is a path summary, closure proof, or direct edge list.

Examples include:

- parent state hash;
- parent semantic hash;
- previous receipt;
- prior build step;
- prior manifest checkpoint.

Causality is the minimal form of lineage.

---

20. Intent Dimension

The Intent dimension declares the action or purpose associated with the state jurisdiction.

Examples:

- build;
- package;
- deploy;
- execute;
- publish;
- distribute;
- verify;
- archive.

Intent is mandatory for execution, deployment, publication, or distribution. It may be absent only when policy permits a non-operative state class.

Intent is not authority.

---

21. Authority Dimension

The Authority dimension contains an attested claim that a particular authority predicate was satisfied.

The verification question is:

$$AuthorityPredicate(d_{authority}, d_{jurisdiction}, P, t, Rev) = \text{true}$$

Authority predicates are policy- and time-dependent by default. They depend on jurisdiction, signer class, time, revocation state, and policy version.

The question is not:

> Who signed this?

This distinction prevents attribution laundering.

---

22. Jurisdiction Dimension

The Jurisdiction dimension specifies the governance domain in which authority is evaluated.

Jurisdictions may be flat, hierarchical, or overlapping. The policy must declare which model applies.

Nested governance systems may treat jurisdiction as a derived dimension whose authority is established through a higher jurisdictional claim.

Examples:

- organization;
- protocol;
- legal regime;
- constitutional rule-set;
- repository policy;
- deployment environment.

The same state may be admissible in one jurisdiction and inadmissible in another.

---

23. Semantics Dimension

The Semantics dimension binds a state jurisdiction to a declared semantic contract.

A semantic contract may include:

- API contract;
- protocol contract;
- interface shape;
- behavioral specification;
- side-effect boundary;
- type-level contract;
- interpretation regime;
- execution promise.

This dimension may be absent where no semantic contract is claimed. Absence must be explicit and policy-evaluable.

---

24. Custody Dimension

The Custody dimension is a provenance claim.

It may record actor continuity, stewardship, possession history, receipt chain, or agent identity.

Custody may be derived from receipts and signatures when the schema declares the derivation rule. Otherwise, it requires its own dedicated attestation type.

Custody is not equivalent to a chain-of-possession proof unless its schema explicitly includes transfer attestations, custody intervals, and signer validation. Custody supports auditability and provenance, but it is not a security guarantee by itself and does not confer authority.

---

25. Resource Dimension

The Resource dimension records declared or measured resource information.

Examples:

- output bytes;
- estimated tokens;
- actual tokens;
- CPU budget;
- memory budget;
- energy budget;
- storage allocation;

- network bandwidth.

Resource claims may be declarative, measured, or both. Runtime measurements may include estimation error or measurement uncertainty. The schema must encode whether each value is estimated, measured, or both, and must define uncertainty semantics where applicable.

A deployment may choose to include resource claims inside the semantic hash or record them as policy-visible metadata. Either choice must be declared.

---

26. Extension Dimensions

Non-canonical dimensions may be added as domain-specific extensions.

Examples include:

- Confidence;
- Clinical Context;
- Consent;
- Regulatory Class;
- Deployment Environment;
- Evidence Level;
- Semantic Reactor State;
- Consensus Weight.

Extension dimensions can be required by policy even if they are not canonical. "Extension" means outside the canonical eight, not necessarily optional.

Extension dimensions must follow the same encoding, attestation, ordering, and policy rules as canonical dimensions.

---

27. Schema Registry

CHC separates ontology from attestation.

Ontology answers:

> What dimensions exist?

Attestation answers:

> What values are claimed?

The schema registry is an append-only, hash-addressed, signed or transparency-logged structure. The schema registry itself is freshness-bound; schema verification must reject stale registry checkpoints under policy.

Each schema entry includes:

- dimension identifier;
- schema hash;
- version;
- machine-checkable migration rule;
- forward compatibility behavior;
- backward compatibility behavior;
- deprecation rule;
- registry signature or log inclusion proof.

Backward compatibility must be explicit. No verifier may silently coerce one schema version into another.

Schema drift is a failure mode, not an implementation detail.

---

28. Freshness Verification

Freshness is an evaluation procedure over existing CHC objects.

A freshness check evaluates:

$$Fresh(H_s, H_m, \mathcal{D}, M, R, P, t, \pi) = \text{true}$$

where:

- M is one or more policy-selected receipt sources and the resulting manifest state;
- R is the trust-root set;
- P is the policy program;
- t is a verifiable epoch, checkpoint, or log position;
- π is freshness evidence.

Policy-selected receipt sources may include local manifests, private or organizational manifest servers, public publisher manifests, peer-to-peer manifest consensus, transparency logs, revocation logs, quorum checkpoints, CRDT state, cache state, checkpoint receipts, consensus receipts, and witness receipts. Signed snapshots are receipt sources only when tied to a specific policy hash or root-set version.

Cached evidence may be used only if the cache is itself freshness-bound by a checkpoint, expiry, signed snapshot, or policy-defined validity window. Cache invalidation is policy-driven; a cache is not an implicit trust root.

The strongest deployment combines independent receipt sources so disagreement becomes evidence rather than noise.

A CHC verifier performs freshness evaluation at the point of attempted action, including program execution. A binary may be byte-identical, semantically

bound, and historically valid, yet still fail execution if the current freshness sources do not establish present admissibility under the active policy.

Freshness sources are backend-neutral. A deployment may use a local manifest, an organizational manifest service, public publisher manifests, independent manifest mirrors, transparency logs, checkpoint witnesses, or a peer-to-peer manifest network. These sources are not treated as authority by themselves. They are evidence sources whose outputs are evaluated by policy.

A stronger deployment can require agreement or non-conflict among many independent manifest sources. For example, users, organizations, auditors, distributions, or public mirrors may publish manifests for software they run and choose to represent as verified by them. A verifier may then require threshold agreement, latest-valid checkpoints, consistency proofs, quorum receipts, or fork-accountability checks across independently operated mirrors.

This creates a practical trust-hardening model: an attacker must not merely compromise a signing key or substitute a local manifest, but must defeat the policy-selected freshness network within the relevant scope. Widely mirrored, independently operated, or peer-to-peer manifest systems make rollback, equivocation, stale binary replay, and unilateral authority substitution substantially harder because disagreement becomes evidence rather than noise.

This does not make public manifests self-authorizing. Each manifest remains an evidence object. Its authority arises only when the active policy accepts it as fresh, consistent, jurisdictionally relevant, and sufficient for execution.

Freshness is not a property of a hash. Freshness is a property of admissibility evaluated against current evidence.

---

29. Freshness Failure Taxonomy

Freshness may fail in distinct ways:

1. **STALE_STATE** — H_s no longer matches the accepted state.
2. **STALE_SEMANTIC_ATTESTATION** — H_m binds old claims no longer accepted.
3. **STALE_REVOCATION_STATE** — revocation information is missing or outdated.
4. **STALE_CHECKPOINT** — the presented log or consensus checkpoint is older than policy permits.
5. **POLICY_STALE** — the verifier is using an outdated policy hash or trust-root interpretation.

A stale policy invalidates the entire decision path. The verifier must reload and re-evaluate under the active policy if available; if the active policy cannot be obtained, strict mode rejects the execution request.

---

30. Freshness Proof Types

A freshness proof π may be one of the following.

Latest Valid Checkpoint Proof

$$\pi_{lvc} = \langle checkpoint, scope, H_P, H_R, sig, consistency_path \rangle$$

Latest Valid Checkpoint Proof is a named proof class in the schema registry and must be independently versioned.

Verification interface:

$$VerifyLVC(H_s, H_m, M, t, \pi_{lvc}, R, P) = \text{true}$$

iff the checkpoint is signed by an accepted checkpoint authority, bound to the active policy hash and root-set hash, current under policy, and consistent with the verifier's highest previously observed checkpoint.

Other proof types include:

1. transparency-log inclusion proof;
2. log consistency proof;
3. consensus checkpoint proof;
4. revocation inclusion or non-inclusion proof;
5. CRDT convergence proof;
6. threshold-signed epoch proof;
7. VDF-backed time proof;
8. secure-time attestation.

Each proof type requires a verification algorithm.

A freshness check succeeds only when the policy-accepted proof set verifies.

---

31. Anti-Rollback and Freeze Resistance

Clients track:

$$checkpoint_{max}$$

the highest valid checkpoint previously observed for a trust domain.

By default, a presented checkpoint is accepted only if:

$$checkpoint_t > checkpoint_{max}$$

Equality-based checkpoint acceptance is allowed only for idempotent rechecks of the same H_s, same H_m, and same H_P within a policy-declared validity window:

$$checkpoint_t = checkpoint_{max}$$

Every accepted checkpoint must include a consistency proof from the prior checkpoint unless the policy defines a trusted genesis condition.

Policies must define maximum staleness:

$$\Delta$$

Execution requires:

$$now - t \leq \Delta$$

or an equivalent consensus-derived freshness bound when local clocks are unreliable.

---

32. Safety and Liveness

CHC separates safety from liveness.

In strict fail-closed mode, unavailable logs, unreachable manifest sources, missing revocation data, or network partitions may block execution. This is a deliberate availability tradeoff.

Deployments may define grace modes, but grace modes must be explicit policy states with bounded duration, recorded risk, auditable refusal/acceptance behavior, and a rollback or revalidation deadline.

Grace mode may bypass staleness only if policy permits it. It should not bypass known revocation. A deployment that permits revocation bypass must declare that behavior as an emergency policy with separate audit treatment.

---

33. Revocation Semantics

CHC distinguishes four revocation classes:

1. **Claim Revocation** — invalidates the admissibility of a specific claim.

2. **Dimension-Value Revocation** — invalidates the encoded value itself under its schema or trust domain.
3. **Signer-Future-Authority Revocation** — prevents a signer from making future claims of a given class.
4. **Trust-Root Revocation** — replaces, narrows, or revokes a trust-root entry or root set.

Claim revocation may cascade into dimension-value revocation when policy declares the claim as constitutive of the value. Dimension-value revocation may cascade into claim revocation when the value is no longer admissible evidence for any claim. Without an explicit cascade rule, the classes remain separate.

A revocation may invalidate:

- only the affected claim;
- only the affected dimension;
- all predicates depending on that dimension;
- the full Semantic Hash;
- all descendants in the lineage DAG.

A revocation proof contains:

- revoked target;
- revocation authority;
- revocation epoch;
- inclusion proof;
- consistency proof;
- propagation bound.

Revocation changes the outcome of freshness and policy evaluation over H_s and H_m. It does not require a third hash.

---

34. Manifest Model

A CHC manifest is an ordered Merkle tree with a root:

$$H_M$$

The manifest tree commits to both leaf content and leaf order using positional indexing or hash-linked sequence commitments, as declared by the manifest schema. Manifest leaf typing is committed in the leaf hash, not only in the positional schema.

A leaf hash has the form:

$$H_{leaf} = H(\texttt{"CHC:MANIFEST:LEAF:v1"} \parallel leaf_type \parallel position \parallel leaf_payload)$$

Manifest leaves may include:

- H_s;
- H_m;
- dimension commitments;
- dimension attestations;
- lineage references;
- policy hash;
- trust-root hash;
- schema hashes;
- freshness references;
- revocation references.

Merkle inclusion proofs allow partial disclosure and partial verification.

---

35. Manifest Consistency and Fork Accountability

Manifest consistency is evaluated within a policy scope.

A policy scope may be:

- package name and version;
- state identity;
- source repository;
- build lineage;
- deployment channel;
- jurisdiction;
- semantic coordinate.

Multiple valid manifests may coexist across different policy scopes over the same state. This is a formal design feature, not a contradiction. Different valid manifests over different scopes do not imply different truth values; they imply different admissibility contexts.

An equivocation proof is:

$$Eq = \langle M_1, M_2, sig_1, sig_2, scope, conflict_predicate \rangle$$

where the conflict occurs within the same policy scope.

Policies may define quarantine, revocation, fork-choice, exclusion, or escalation.

---

36. Policy Function

Execution is allowed iff:

$$P(H_s, H_m, \mathcal{D}, M, R, t, \pi) = \text{true}$$

P must be deterministic, total, versioned, auditable, hash-addressed, and explicit about fail-closed or grace behavior.

The policy hash is:

$$H_P = H(\texttt{"CHC:POLICY:v1"} \parallel policy_bytes)$$

Freshness sources must bind H_P or otherwise prove that their evidence is valid for the policy being evaluated.

A recommended precedence order is:

1. schema validity;
2. trust-root validity;
3. signature and attestation verification;
4. revocation;
5. staleness;
6. manifest consistency;
7. partial verification;
8. semantic equivalence transformation, only if explicitly enabled by policy;
9. execution decision.

A policy bug is a first-class failure mode.

---

37. Partial Verification

Partial verification is unsafe by default.

Let:

$$\mathcal{D}' \subset \mathcal{D}$$

be the verified dimension subset.

Let:

$$M_{missing} = \mathcal{D} \setminus \mathcal{D}'$$

be the missing dimension set.

Partial verification is permitted only when both are explicitly declared:

1. the required core dimension set;
2. the missing-dimension risk envelope.

Omitted dimensions may still be implicitly required through transitive policy dependencies. For example, omitting Jurisdiction may invalidate Authority because the authority predicate may depend on jurisdictional scope.

A policy may define mandatory core dimensions:

$$\mathcal{D}_{core} \subseteq \mathcal{D}$$

Execution is denied unless:

$$\mathcal{D}_{core} \subseteq \mathcal{D}'$$

Non-core omissions may be accepted only under a declared risk envelope. Partial verification must emit an explicit risk class, not merely pass or fail.

---

38. Semantic Equivalence

Semantic equivalence is a policy-specified optimization, not a general property of CHC.

Define a schema-declared projection:

$$proj_S : \mathcal{D} \to S$$

and a schema-version-specific equivalence relation:

$$\sim_{S,v}$$

Two CHC objects are semantically equivalent under (S) and schema version (v) iff:

$$proj_S(\mathcal{D}^{(1)}) \sim_{S,v} proj_S(\mathcal{D}^{(2)})$$

Equivalence is checked over normalized values, projections, or schema-defined relations. The relation must be explicit and policy-enabled. Schema-version-specific equivalence prevents silent cross-schema comparison.

---

39. Semantic Continuity

CHC separates state identity from semantic identity, but practical systems often require semantic continuity across time.

Given two state jurisdictions:

$$j_{t_0}, j_{t_1}$$

with:

$$H_s(j_{t_0}), H_s(j_{t_1})$$

and semantic hashes:

$$H_m(t_0), H_m(t_1)$$

a future CHC extension may define semantic continuity as a policy-evaluable relation between the two semantic states.

At minimum, semantic continuity would ask:

```
Which semantic claims persisted?
Which claims changed?
Which claims were revoked?
Which authority predicates remain admissible?
Which lineage edges justify the transition?
```

CHC does not require a complete theory of semantic evolution to function. It requires only that semantic claims be bound to state and evaluated under policy. However, systems such as DKN and CSH naturally pressure this boundary, because knowledge identity and constitutional meaning often depend not merely on a single semantic hash but on admissible semantic evolution across time.

Semantic continuity is therefore a natural future extension of CHC: the study of how semantic identity persists, changes, or fails across successive State Jurisdictions.

---

40. Privacy and Selective Disclosure

Rich semantic dimensions may leak sensitive information.

CHC supports commitment-based dimensions:

$$C_i = Commit(value_i, r_i)$$

Verification may use:

- opening proofs;
- Merkle inclusion proofs;
- set-membership proofs;
- zero-knowledge predicates.

Commitments do not reveal raw values, but schema membership disclosure, reused commitments, stable identifiers, repeated signer keys, or key reuse across policy scopes may leak equality or linkability. Policies must account for this.

---

41. Source-Binary Binding and Build Proofs

CHC binds source to binary through a two-phase architecture.

Phase 1: Clean Build

The build system produces:

$$Binary_{clean}$$

and computes:

$$H_s(Binary_{clean})$$

Native CHC deployments require:

- hermetic builds;
- reproducibility evidence;
- build sandboxing;
- dependency pinning;
- environment hashing;
- independent rebuild checks where policy requires them.

Phase 2: Semantic Binding

Semantic dimensions are attached after H_s is fixed.

The build proof:

$$\pi_{build}$$

is included in the Causal dimension.

It must bind at minimum:

- source hash;

- clean binary hash;
- environment hash;
- dependency graph or dependency manifest;
- builder identity;
- build step evidence.

Independent rebuild checks strengthen π_{build}, but the proof must bind source and environment even when such checks are omitted.

Without π_{build}, source-binary correspondence remains an attested claim rather than a reproducibility proof.

---

42. Recursive Toolchain Binding and Trusting Trust

CHC addresses Thompson-style "Trusting Trust" attacks by extending the same state-and-semantic binding applied to application binaries to the compiler toolchain itself. A compiler is treated as a CHC state jurisdiction with its own State Hash, Semantic Hash, causal lineage, build proof, source hash, environment hash, dependency graph, compiler provenance, receipts, and freshness evidence.

The compiler used to build an application is therefore not an ambient trusted premise; it becomes a policy-evaluable dependency.

The recursive dependency shape is:

```
application binary
  depends on compiler binary
    depends on compiler source
      depends on prior compiler/toolchain state
        depends on receipts, rebuilds, policy, freshness
```

CHC does not merely ask:

```
Did this source produce this binary?
```

It asks:

```
What produced the producer?
```

CHC recursively converts trusted premises into auditable jurisdictions. Every producer may itself become a state jurisdiction subject to provenance, receipts, and admissibility evaluation. For toolchains, execution-time freshness means that a compiler binary is not accepted merely because it was once valid or locally present. At the point of use, the verifier may require current admissibility evidence from policy-selected manifest backends, including independent mirrors or peer-to-peer manifest consensus. This turns stale or forked compiler lineages into freshness failures rather than invisible trust assumptions.

Trusting Trust is defeated as an invisible attack class within a toolchain-closed CHC verification boundary. CHC does not make subversion impossible; it

prevents operative producers inside the declared boundary from remaining unaudited premises. A compiler subversion attempt must either satisfy recursive admissibility requirements or surface as a state, lineage, receipt, freshness, rebuild, dependency, or policy failure. Components outside the declared boundary remain explicit trusted premises until modeled as CHC State Jurisdictions.

Within the declared CHC verification boundary, Thompson-style compiler subversion becomes an auditable and rejectable provenance failure rather than an invisible assumption. This boundary may include the compiler, assembler, linker, build scripts, dependency graph, build environment, operating system image, container image, or any other toolchain state jurisdiction that policy requires. Hardware, microcode, firmware, policy engines, and external infrastructure remain within scope only when modeled as CHC state jurisdictions with their own semantic attestations and freshness checks.

42.1 No Ambient Producer Principle

Trusting Trust attacks persist because conventional build systems allow producers to remain ambient premises. A compiler, assembler, linker, build script, dependency resolver, package manager, runtime, signing tool, or registry writer may influence executable state without itself being represented as an admissible object in the same verification system.

CHC rejects ambient producer privilege.

Any artifact, tool, service, or environment capable of producing, transforming, lowering, assembling, linking, packaging, signing, registering, admitting, or executing a State Jurisdiction must itself be one of:

```
1. a CHC State Jurisdiction with its own state hash, semantic hash,
receipts, lineage, freshness evidence, and admissibility predicate;
or

2. an explicitly declared trusted premise outside the CHC
verification boundary.
```

No producer may silently occupy a privileged position between state identity and admissibility.

This principle can be stated compactly:

```
No executable state may be admitted from an unaudited producer
inside the declared verification boundary.
```

A compiler is therefore not merely a tool that produces evidence. It is itself evidence-bearing state. Its authority to produce executable artifacts is derived from admissibility, not from its role, name, institutional origin, signature, or historical acceptance.

42.2 Toolchain Closure

A CHC verification boundary is toolchain-closed when every producer participating in the build or admission of executable state is itself modeled as a CHC State Jurisdiction or explicitly declared as an out-of-bound trusted premise.

A toolchain-closed boundary may include:

```
source repository
compiler source
compiler binary
assembler
linker
optimizer
code generator
build scripts
dependency resolver
package manager
container image
operating system image
runtime
policy engine
registry writer
signing tool
receipt generator
```

Each modeled producer has:

```
state hash
semantic hash
source hash where applicable
build proof
environment hash
dependency graph
authority dimension
jurisdiction dimension
custody dimension
receipts
freshness evidence
policy-bound admissibility
```

A producer outside the closure boundary is not ignored. It is recorded as an explicit trusted premise with a declared trust downgrade, risk class, and policy-visible nonclaim.

42.3 Trusting Trust Closure Claim

Within a toolchain-closed CHC verification boundary, a Thompson-style compiler-subversion attack cannot remain an invisible assumption.

A malicious compiler that produces a compromised binary while presenting benign source must satisfy the same admissibility predicates as any other executable producer. If the compiler, its source, its prior compiler, its build environment, its dependencies, or its build receipts are missing, stale, revoked, inconsistent, non-reproducible where reproducibility is required, or inconsistent with independent rebuild evidence, execution is refused.

Thus, inside the declared CHC boundary, Trusting Trust is transformed from an invisible trust premise into a policy-evaluable failure class.

A Trusting Trust attack must appear as at least one of:

```
state hash mismatch
semantic hash mismatch
missing producer jurisdiction
missing source-binary build proof
invalid causal lineage
unmodeled prior compiler
unmodeled build environment
dependency drift
stale compiler authority
revoked toolchain authority
failed independent rebuild
failed diverse rebuild
receipt mismatch
policy-source inconsistency
freshness failure
explicit out-of-bound trusted premise
```

CHC therefore does not claim that all possible subversion is metaphysically impossible. It claims that no producer inside the declared verification boundary may remain both operative and unaudited.

This is the precise sense in which CHC counters Trusting Trust:

```
Trusting Trust is defeated as an invisible attack class
within a toolchain-closed CHC verification boundary.
```

Components outside the declared boundary remain trusted premises unless and until they are admitted as CHC State Jurisdictions. Hardware, microcode, firmware, physical manufacturing, and external infrastructure are not magically solved by CHC; they become further boundary candidates for recursive admission.

42.4 Relation to Diverse Double-Compiling

Diverse Double-Compiling strengthens CHC toolchain closure by introducing an independent producer path. CHC does not replace diverse rebuild techniques; it gives them jurisdiction, receipts, freshness, and admissibility semantics.

A CHC policy may require:

```
compiler_A(source) -> binary_A

compiler_B(source) -> binary_B

Compare(binary_A, binary_B) -> verdict
```

The comparison verdict need not always be byte identity. Depending on the target, policy may accept:

```
BYTE_IDENTICAL
OUTPUT_CORRESPONDS
RECEIPT_CORRESPONDS
STRUCTURE_CORRESPONDS
NOT_COMPARABLE
GAP_CERTIFIED
REFUSED
```

Where byte identity is required and fails, execution is refused.

Where byte identity is not expected, CHC requires explicit policy justification for accepting weaker correspondence. The verifier must not silently promote behavioral correspondence into equivalence.

This lets CHC generalize the insight of Diverse Double-Compiling: independent production paths become receipt-bearing evidence inside a larger admissibility calculus.

42.5 Compiler Admissibility Predicate

A compiler is admissible to produce executable state only if policy accepts its producer lineage.

A minimal compiler admissibility predicate is:

```
CompilerAdmissible(C, P) =
  StateHashValid(C)
  and SemanticHashValid(C)
  and SourceHashBound(C)
  and BuildProofValid(C)
  and EnvironmentHashBound(C)
  and DependencyGraphAdmissible(C)
  and ProducerLineageClosedOrDeclared(C)
  and FreshnessValid(C, P)
  and AuthorityPredicate(C, P)
  and NoUnmodeledProducer(C)
```

An application binary produced by compiler C is admissible only if:

```
ApplicationAdmissible(A, C, P) =
  StateHashValid(A)
```

```
  and SemanticHashValid(A)
  and SourceBinaryProofValid(A, C)
  and CompilerAdmissible(C, P)
  and BuildReceiptsValid(A)
  and FreshnessValid(A, P)
  and AuthorityPredicate(A, P)
```

This makes the compiler an admissible dependency rather than a background assumption.

---

43. Independence Requirement

CHC requires hash-domain independence plus build-pipeline isolation between State Hash generation and semantic dimension binding: no semantic feedback into Phase 1 inputs.

This means:

1. Phase 1 build input excludes semantic dimensions;
2. metadata embedding occurs only after H_s is computed;
3. rebuilding with altered semantic dimensions does not alter H_s;
4. final artifact metadata can be removed or normalized to recover the clean binary hash.

This prevents circular hash dependency and semantic feedback channels.

---

44. Worked Example: Ledger-Tool v2.4

This example illustrates how CHC distinguishes state identity, semantic identity, and admissibility.

Assume an artifact:

```
name: ledger-tool
version: v2.4
role: release_binary
channel: stable
```

44.1 State Jurisdiction

The declared State Jurisdiction is the release binary payload:

```
J = ledger-tool:v2.4:release_binary
```

The verifier computes:

```
H_s = sha256("ledger-tool-v2.4-clean-binary-bytes")
    = 7d9c...a41
```

This answers:

```
What exists?
```

It does not answer who may publish it, whether it is fresh, or whether it may execute.

44.2 Semantic Dimensions

The artifact carries the following semantic dimensions:

```
Artifact State and Encoding:
  state_hash: 7d9c...a41
  media_type: application/x-elf
  serialization_format: elf64
  normalization_scheme: chc-clean-binary-v1
  artifact_role: release_binary

Intent:
  publish
  execute

Jurisdiction:
  ledger-tool/stable-channel

Authority:
  maintainer-quorum-2-of-3

Causal:
  source_hash: b91e...119
  build_environment_hash: 3ad2...92f
  dependency_manifest_hash: 0be7...8cd
  build_receipt: receipt:build:812

Custody:
  builder: ci-runner-04
  release_steward: maintainer-set-alpha
```

Each dimension value is normalized, hashed, and attested.

The Semantic Hash is:

```
H_m = H("CHC:H_m:v1" || H_s || H_R || H_P || E(D))
    = 4aa2...c90
```

This answers:

```
What is claimed about what exists?
```

It does not by itself prove the claims are true. It binds the claims to the state.

44.3 Receipts and Freshness

The verifier receives:

```
build receipt:
  receipt:build:812

maintainer quorum receipt:
  receipt:quorum:421

stable-channel checkpoint:
  checkpoint:812

revocation checkpoint:
  revocation-log:812

policy hash:
  H_P = 991d...e02

trust-root hash:
  H_R = 61ff...772
```

The freshness proof states:

```
pi_lvc = latest-valid-checkpoint(
  checkpoint = 812,
  scope = ledger-tool/stable-channel,
  H_P = 991d...e02,
  H_R = 61ff...772
)
```

The policy evaluates:

```
P(H_s, H_m, D, M, R, t, pi)
```

and checks:

```
state hash matches stable manifest
semantic hash matches release manifest
maintainer quorum is valid
build receipt is valid
source-binary build proof is valid
revocation state is current
checkpoint is latest valid checkpoint
policy hash is current
```

Result:

```
ADMISSIBLE
```

The artifact may execute under this policy.

44.4 Same State, Revoked Authority

Now assume the binary is unchanged:

```
H_s = 7d9c...a41
```

and the Semantic Hash remains historically valid:

```
H_m = 4aa2...c90
```

But one maintainer key in the quorum is revoked at checkpoint 819:

```
revocation-log:819
  revoked: maintainer-key-B
  scope: ledger-tool/stable-channel
```

The verifier evaluates again under the current checkpoint:

```
pi_lvc = latest-valid-checkpoint(
  checkpoint = 819,
  scope = ledger-tool/stable-channel,
  H_P = 991d...e02,
  H_R = 61ff...772
)
```

The State Hash check succeeds.

The Semantic Hash check succeeds.

The freshness and revocation checks fail the authority predicate:

```
AuthorityPredicate(
  maintainer-quorum-2-of-3,
  ledger-tool/stable-channel,
  P,
  checkpoint 819,
  Rev
) = false
```

Result:

```
INADMISSIBLE
```

The artifact is not denied as a historical object.

The claim is not erased.

The binary is not rewritten.

The execution decision changes because admissibility changed.

This is the Admissibility Separation Principle in operation:

```
same state
same historical semantic claims
different admissibility result
```

---

45. Concrete Threat Example

An attacker compromises the build server for `ledger-tool v2.4` and publishes a malicious binary through a legitimate distribution channel.

Decisive failure:

1. The verifier computes H_s from the downloaded binary.
2. The current accepted manifest for `ledger-tool v2.4` records a different H_s.
3. The artifact is refused.

Corroborating failures if the attacker supplies a replacement manifest:

- replacement manifest not accepted by freshness sources;
- Authority dimension lacks a valid predicate;
- Causal dimension lacks a build proof from certified source.

Result:

$$P(H_s, H_m, \mathcal{D}, M, R, t, \pi) = \text{false}$$

Execution is refused.

---

46. Stale Binary Example

An attacker presents an old but once-valid binary.

The State Hash check succeeds.

The Semantic Hash check succeeds.

However, the freshness check fails because the presented checkpoint is older than the highest previously observed checkpoint, the artifact exceeds the policy staleness window, or revocation state has advanced.

Therefore:

$$Fresh(H_s, H_m, \mathcal{D}, M, R, P, t, \pi) = \text{false}$$

and execution is denied.

Historical validity is a prerequisite, but not sufficient on its own, for present admissibility.

---

47. Native CHC and Retrofit CHC

Native CHC

An artifact is built with CHC from the beginning. Its State Hash, Semantic Hash, lineage, manifests, and freshness evidence are generated as part of its lifecycle.

Retrofit CHC

A legacy artifact may receive semantic dimensions after creation.

Retrofit dimensions must include machine-readable, schema-validated assurance fields:

```
required schema_field: chc_origin = post_hoc
required trust_downgrade: true
```

Post-hoc dimensions are lower assurance unless anchored to a contemporaneous source of truth such as an archived release manifest, timestamped transparency log, reproducible build record, or independently verifiable distribution record.

---

48. Interoperability

CHC composes with existing verification systems while relocating their role within the trust architecture. Signing systems, transparency systems, provenance systems, update systems, and registry systems remain valuable evidence producers. CHC does not require their institutional authority to be treated as self-justifying. Their outputs become policy-evaluable receipts within a common admissibility framework.

CHC subsumes rather than replaces these systems. Their cryptographic outputs remain useful, but their authority is normalized into evidence participating in admissibility evaluation.

OCI, Container Images, and Container Image Registries

CHC manifests may be attached as OCI artifact metadata, linked attestations, registry-adjacent verification records, or container image registry policy artifacts.

Artifact and Package Registries

CHC can bind package registry metadata, release artifacts, package indexes, and binary distribution records into typed dimensions and freshness sources.

SBOM

SPDX or CycloneDX documents may be bound as dimensions or lineage leaves.

in-toto

in-toto attestations may populate build-proof and lineage dimensions.

SLSA

SLSA provenance may be encoded as build-quality dimensions.

Sigstore

Sigstore identities and transparency logs may serve as identity and log inputs.

TUF

TUF timestamp and targets metadata may serve as freshness and manifest-consistency inputs.

---

49. Comparison to Existing Systems

CHC is a composition and authority-normalization layer. Existing systems continue to produce signatures, attestations, checkpoints, manifests, and transparency records. CHC changes how those outputs are interpreted by reducing them to evidence objects subject to policy evaluation. Authority emerges from admissibility rather than institutional position.

Institutional assertions retain evidentiary value but lose privileged status outside the admissibility calculus.

CHC's unique claim is policy-evaluable composition across dimensions, attestations, manifests, revocation state, and freshness sources.

Existing systems primarily answer whether a claim was made. CHC additionally asks whether the claim remains admissible under current policy, current provenance state, and current freshness conditions.

TUF provides secure update metadata, including timestamp and target metadata. CHC can consume TUF metadata as freshness and manifest evidence while adding typed semantic dimensions and per-dimension attestations.

in-toto provides supply-chain layout and link metadata. CHC can bind in-toto attestations into lineage and build-proof dimensions while adding semantic hashing and policy evaluation over typed dimensions.

SLSA defines supply-chain assurance levels. CHC can encode SLSA level and provenance as dimensions while adding cryptographic semantic composition.

Sigstore provides signing, identity, and transparency logging. CHC can use Sigstore as an identity and log input while adding governance algebra, multidimensional semantic binding, and authority/jurisdiction separation.

The central reviewer question for CHC is whether existing systems already provide equivalent separation under different terminology. CHC's answer is that existing systems often provide elements of the separation, but do not generally make State Identity, Semantic Identity, and Admissibility independently addressable domains under a unified cryptographic admissibility primitive.

---

50. Complexity Analysis

Let:

$$k = |\mathcal{D}|$$

and let s_i be the number of signatures or attestation elements for dimension i.

Dimension storage is:

$$O(k)$$

Signature storage is:

$$O(\sum_i s_i)$$

Dimension verification is:

$$O(k)$$

Signer verification cost is:

$$O(\sum_i s_i)$$

unless aggregation reduces it.

Trust-root validation cost is:

$$C_R$$

depending on the root model. Flat root-set lookup may be $O(1)$ or $O(\log r)$; path validation in hierarchical models may scale with chain length.

Policy evaluation cost is:

$$C_P$$

relative to policy program size, referenced inputs, and required external checks.

Policy-source verification cost is:

$$C_{PS}$$

and may dominate latency when freshness sources, policy registries, and trust-root registries must be queried or reconciled.

Merkle proof size is logarithmic in the number of manifest leaves:

$$O(\log N)$$

not necessarily logarithmic in the number of semantic claims unless each claim is separately represented as a leaf.

Freshness verification cost depends on proof type:

- log inclusion: $O(\log N)$;
- consistency proof: $O(\log N)$;
- threshold checkpoint: $O(s)$ or $O(1)$ with aggregation;
- CRDT convergence: implementation-dependent;
- P2P manifest consensus: network-dependent.

Worst-case bandwidth depends on manifest source count, proof size, revocation proof size, trust-root fanout, cache consistency proofs, and policy evidence requirements.

---

51. Failure Modes

CHC implementations must explicitly handle:

- missing dimension;
- conflicting authorities;
- stale manifest;

- stale cache with valid signature;
- unavailable log;
- partitioned network;
- revocation conflict;
- schema mismatch;
- schema drift;
- policy bug;
- policy downgrade;
- misconfigured policy source;
- policy-source inconsistency;
- operator misconfiguration;
- inconsistent cache;
- compromised signer;
- compromised trust root;
- replayed manifest;
- equivocated manifest;
- registry spoofing;
- schema rollback;
- privacy leakage through reused commitments.

Default behavior should be fail-closed unless policy declares a bounded grace mode.

---

52. CHC-Derived Architectures

The CHC architecture establishes a generalized cryptographic pattern:

1. fingerprint a bounded State Jurisdiction;
2. bind semantic claims to that state;
3. evaluate admissibility under policy and freshness.

Several downstream systems may be understood as specializations of this pattern.

Distributed Knowledge Networks (DKN) apply State Jurisdiction fingerprinting to self-certifying knowledge identity, unsquattable namespace construction, distributed provenance tracking, and policy-evaluable knowledge state continuity. Knowledge identities become self-certifying and namespace ownership becomes cryptographically unsquattable through jurisdiction-bound identity construction.

Constitutional Semantic Hashing (CSH) applies the Semantic Hash concept to governance-bound semantic identity and policy-evaluable meaning claims. Meaning becomes policy-evaluable rather than authority-asserted, extending admissibility principles into semantic governance.

These systems are not required by CHC and are not equivalent to CHC. They are examples of architectures derived from the same underlying cryptographic separation of state identity, semantic identity, and admissibility.

53. Empirical Evaluation Plan

CHC should be evaluated against benchmark scenarios:

1. compromised build server;
2. malicious package substitution;
3. stale valid binary replay;
4. manifest equivocation;
5. revoked authority key;
6. forked manifest network;
7. partial verification under constrained runtime;
8. retrofit CHC over legacy binaries;
9. schema drift attack;
10. policy bug or policy downgrade;
11. partial disclosure privacy leakage;
12. selective-disclosure reidentification or linkability leakage;
13. equivalence-class abuse or overbroad semantic equivalence;
14. compiler/toolchain subversion under recursive CHC binding.

Metrics:

- false acceptance rate;
- false refusal rate;
- detection latency;
- revocation propagation time;
- verification overhead;
- manifest bandwidth;
- consensus latency;
- developer integration cost;
- privacy leakage under repeated selective disclosure;
- linkability across policy scopes;
- toolchain subversion detection rate.

Baselines:

- TUF;
- in-toto;
- SLSA;
- Sigstore;
- OCI signatures;
- SBOM-only workflows;
- reproducible-build-only workflows.

54. Conclusion

CHC is a cryptographic architecture for binding state identity to semantic attestations without confusing the two.

It does not prove meaning.

It cryptographically binds claims about meaning.

It does not make signatures into authority.

It separates identity from admissibility.

It does not treat historical validity as present trust.

It evaluates freshness at execution time using the State Hash, Semantic Hash, manifests, revocation state, checkpoints, caches, receipts, and policy.

The core contribution is:

$$H_s \rightarrow H_m$$

plus a freshness verification procedure.

State identity.

Semantic attestation.

Present admissibility under policy.

CHC in one sentence:

`Traditional cryptographic systems bind state.`

`CHC binds state, claims, and admissibility as separately addressable cryptographic domains.`

Together they form a candidate architectural foundation for a family of governed execution systems, including verified compilation, supply-chain security, semantic operating systems, distributed knowledge networks, constitutional governance platforms, and continuous execution governance.

CHC extends content addressing into admissibility addressing.

Traditional cryptographic systems answer:

What exists?

CHC answers:

What exists? What is claimed? What may act?

The transition from identity to admissibility is the transition from cryptographic integrity to cryptographic governance. Governance refers to deterministic admissibility evaluation over evidence and policy, not institutional or political governance.

In this sense, CHC treats governance not as an external layer imposed upon cryptography, but as a cryptographically addressable object in its own right.

References

Author's Related Work Adam Ableman. Related Works: Cryptographic Homological Compilation; Authority Laundering Theory; Distributed Knowledge Networks (DKN); Constitutional Semantic Hashing (CSH); Governed Semantic Operating System (GSOS).

Cryptographic Foundations, Hashing, and Time-Stamping Haber, Stuart, and W. Scott Stornetta. "How to Time-Stamp a Digital Document." *Journal of Cryptology*, vol. 3, no. 2, 1991, pp. 99–111.

Merkle, Ralph C. "Protocols for Public Key Cryptosystems." *Proceedings of the 1980 IEEE Symposium on Security and Privacy*, 1980, pp. 122–133.

Rivest, Ronald L. "The MD5 Message-Digest Algorithm." *RFC 1321*, Internet Engineering Task Force, April 1992.

National Institute of Standards and Technology. *Secure Hash Standard (SHS).* FIPS PUB 180-4, August 2015.

National Institute of Standards and Technology. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions.* FIPS PUB 202, August 2015.

Nakamoto, Satoshi. "Bitcoin: A Peer-to-Peer Electronic Cash System." 2008.

Distributed Systems, Ordering, and Trust Lamport, Leslie. "Time, Clocks, and the Ordering of Events in a Distributed System." *Communications of the ACM*, vol. 21, no. 7, 1978, pp. 558–565.

Lampson, Butler W. "Protection." *Proceedings of the 5th Princeton Conference on Information Sciences and Systems*, 1971.

Trusting Trust, Reproducible Builds, and Compiler Provenance Thompson, Ken. "Reflections on Trusting Trust." *Communications of the ACM*, vol. 27, no. 8, 1984, pp. 761–763.

Wheeler, David A. "Countering Trusting Trust through Diverse Double-Compiling." *Proceedings of the 21st Annual Computer Security Applications Conference*, 2005.

Wheeler, David A. *Fully Countering Trusting Trust through Diverse Double-Compiling.* Ph.D. dissertation, George Mason University, 2009.

Reproducible Builds Project. *Reproducible Builds: Independently-Verifiable Path from Source to Binary Code.*

Software Supply Chain Security and Provenance The Update Framework Project. *The Update Framework Specification.*

Samuel, Justin, Nick Mathewson, Justin Cappos, and Roger Dingledine. "Survivable Key Compromise in Software Update Systems." *Proceedings of the 17th ACM Conference on Computer and Communications Security*, 2010.

Torres-Arias, Santiago, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. "in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes." *Proceedings of the 28th USENIX Security Symposium*, 2019.

in-toto Project. *in-toto Framework.*

SLSA Project. *Supply-chain Levels for Software Artifacts Specification.*

Sigstore Project. *Sigstore: Software Signing for Everybody.* Cooper, Zachary, et al. "Sigstore: Software Signing for Everybody." *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022.

Rekor Project. *Rekor Transparency Log.*

Transparency Logs, Key Transparency, and Verifiable Data Structures Laurie, Ben, Adam Langley, and Emilia Kasper. *Certificate Transparency.* RFC 6962, Internet Engineering Task Force, 2013.

Laurie, Ben, Adam Langley, Emilia Kasper, Eran Messeri, and Rob Stradling. *Certificate Transparency Version 2.0.* RFC 9162, Internet Engineering Task Force, 2021.

Google Trillian Project. *Trillian: A Transparent, Highly Scalable, and Cryptographically Verifiable Data Store.*

Melara, Marcela S., Aaron Blankstein, Joseph Bonneau, Edward W. Felten, and Michael J. Freedman. "CONIKS: Bringing Key Transparency to End Users." *Proceedings of the 24th USENIX Security Symposium*, 2015.

Content Addressing, Artifacts, and Package Metadata Benet, Juan. "IPFS — Content Addressed, Versioned, P2P File System." 2014.

Open Container Initiative. *OCI Image Format Specification.*

Open Container Initiative. *OCI Distribution Specification.*

Open Container Initiative. *OCI Artifact Specifications and Guidance.*

SPDX Project. *System Package Data Exchange Specification.*

ISO/IEC. *ISO/IEC 5962:2021 — SPDX Specification.*

OWASP Foundation. *CycloneDX Bill of Materials Specification.*

Ecma International. *ECMA-424: CycloneDX Bill of Materials Specification.*

Identity, Credentials, and Attestation Models World Wide Web Consortium. *Verifiable Credentials Data Model v2.0.*

World Wide Web Consortium. *Verifiable Credentials Overview.*

Adams, Carlisle, and Steve Lloyd. *Understanding PKI: Concepts, Standards, and Deployment Considerations.* Addison-Wesley, 2002.

Software Release, Governance, and Component Systems van der Hoek, André, and Alexander L. Wolf. "Software Release Management for Component-Based Software." *Software: Practice and Experience*, vol. 33, no. 1, 2003, pp. 77–98.

Foundations and Working Groups Open Source Security Foundation (OpenSSF). *OpenSSF Best Practices and Project Specifications.* Linux Foundation.

Cloud Native Computing Foundation (CNCF). *CNCF Project Documentation and Technology Radar.*.

Corpus-Wide Reference Positioning

This paper argues that cryptographic verification systems typically bind artifacts to hashes, signatures, attestations, or provenance records. Its nearest external comparison fields are crypto, security, governance. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: A valid hash, signature, or inclusion proof establishes a bounded cryptographic relation, not meaning, authority, safety, or jurisdiction by itself. Unforgeable capabilities can constrain exercise

of authority while still leaving allocation, delegation, revocation, and social legitimacy incorrectly specified. A written constitution can legitimate domination as easily as constrain it unless amendment, participation, enforcement, and appeal remain effective in practice. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Anders Rundgren, Bret Jordan, and Samuel Erdtman. *JSON Canonicalization Scheme (JCS).* RFC 8785, 2020. https://www.rfc-editor.org/rfc/rfc8785 **Relationship:** Adjacent canonicalization standard relevant to stable cryptographic commitments.

[**E2**] Ben Laurie et al. *Certificate Transparency Version 2.0.* RFC 9162, 2021. https://www.rfc-editor.org/rfc/rfc9162 **Relationship:** Adjacent append-only Merkle-log standard for inclusion and consistency evidence.

[**E3**] Manu Sporny et al., eds. *Decentralized Identifiers (DIDs) v1.0.* W3C Recommendation, 19 July 2022. https://www.w3.org/TR/did-core/ **Relationship:** Adjacent decentralized identity standard; identifier resolution is not assumed to settle semantic standing or jurisdiction.

[**E4**] Juan Benet. "IPFS - Content Addressed, Versioned, P2P File System." 2014. arXiv:1407.3561. https://arxiv.org/abs/1407.3561 **Relationship:** Predecessor content-addressed network architecture; content identity is compared with, not treated as sufficient for, semantic jurisdiction.

[**E5**] Jerome H. Saltzer and Michael D. Schroeder. "The Protection of Information in Computer Systems." *Proceedings of the IEEE* 63(9), 1975, pp. 1278-1308. DOI: 10.1109/PROC.1975.9939. https://doi.org/10.1109/PROC.1975.9939 **Relationship:** Foundational protection principles used as a comparison point for least authority, mediation, and inspectability.

[**E6**] Mark S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control.* PhD dissertation, Johns Hopkins University,

2006. http://www.erights.org/talks/thesis/ **Relationship:** Predecessor object-capability treatment of authority and compositional security.

[**E7**] Gerwin Klein et al. “seL4: Formal Verification of an OS Kernel.” *SOSP '09*, 2009, pp. 207-220. DOI: 10.1145/1629575.1629596. https://doi.org/10.1145/1629575.1629596 **Relationship:** Predecessor for machine-checked implementation correspondence and capability-based kernel design.

Ableman corpus

[**A1**] Adam Ableman. *The Three Hash Rule*. Ableman Corpus current edition; work identity `PAPER.PUB.20788617`;
source SHA-256
`F6FA73D083B70711A8C817F7779751D1506FEBA0E356DB9805BB1570663731B5`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Constitutional Semantic Hashing: Governed Commitment Schemes for Semantic Provenance, Admissibility, and Constitutional Continuity*. Ableman Corpus current edition; work identity `PAPER.PUB.20248734`;
source SHA-256
`B1DAC3E2CD4D57BAFBF6462BA6ACFFF7CF120CDB7FA37D0C173525744CC0A2F0`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Self-Certifying Proof-Carrying Identity for Registrar-Free Namespace Networks*. Ableman Corpus current edition; work identity `PAPER.PUB.20114888`;
source SHA-256
`1AFDBD390D3BF28224727FE150FFB30CA1810D997778CC8702B6CD20C4FA5B45`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Semantic Hashing: Hashes as Declared Equivalence Classes of Meaning*. Ableman Corpus current edition; work identity `PAPER.PUB.18500533`;
source SHA-256
`7DA63A76DDF492FFC289F486923FB430879DC148CF3BB16EB0E755E2D4AB9C14`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Effect Sovereignty*. Ableman Corpus current edition; work identity `PAPER.PUB.18737323`;
source SHA-256
`1DB8A34946A8705F6F4BA4FC3B12B251FC7C83DBDF217F9DDFF55529C8612EE7`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Receipt-Bound Capabilities: Membrane Tokens for Governed Interaction with Reality.* Ableman Corpus current edition; work identity `PAPER.PUB.18601631`;
source SHA-256
`6DE46F821594AB11BFB7F8373368C452BD89E5BD4EB172B88D9467E87CF74C33`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *The Integrity Enforcement Layer.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-505307E9E6571B63`;
source SHA-256
`F858AB32E0937B796353CAE573DD4183FDF4F179A3E1E7E40C4E0C6BF1AB0018`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.20602448